CSI INSTITUTE OF TECHNOLOGY, THOVALAI

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CCS356/OBJECT ORIENTED SOFTWARE ENGINEERING

UNIT I

SOFTWARE PROCESS AND AGILE DEVELOPMENT

Introduction to Software Engineering, Software Process, Perspective and Specialized Process Models -Introduction to Agility-Agile process-Extreme programming-XP Process-Case Study.

1.1 Introduction to Software Engineering

1.1 Software

The product that software professionals build and then support over the long term. Software encompasses:

(1) Instructions (computer programs) that when executed provide desired features, function, and performance;

(2) Data structures that enable the programs to adequately store and manipulate information.

(3) Documentation that describes the operation and use of the programs.

1.1.1 Features of Software

Its characteristics that make it different from other things human being build.

1.1.2 Features of such logical system

• Software is developed or engineered; it is not manufactured in the classical sense which has quality problem.

• Software doesn't "wear out.‖ but it deteriorates (due to change). Hardware has bathtub curve of failure rate (high failure rate in the beginning, then drop to steady state, then cumulative effects of dust, vibration, abuse occurs). •

• Modern reusable components encapsulate data and processing into software parts to be reused by different programs. E.g. graphical user interface, window, pull-down menus in library etc.

1.1.3 Software Engineering Definition

Software Engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.

1.1.4 A Layered Technology

Software engineering is a layered technology. The bedrock that supports software engineering is a quality focus. The foundation for software engineering is the process layer. The software engineering process is the glue that holds the technology layers together and enables rational and timely development of computer software.

Process defines a frame work that must be established for effective delivery of software engineering technology. Software engineering methods provide the technical how-to's for building software. Methods encompass a broad array of tasks that include communication, requirements analysis, design modelling, program construction, testing, and support.

Software engineering methods rely on a set of basic principles that govern each area of the technology and include modelling activities and other descriptive techniques. Software engineering tools provide automated or semi-automated support for the process and the methods.

Layers of Software Engineering

When tools are integrated so that information created by one tool can be used by another, a system for the support of software development, called computer-aided software engineering, is established

1.2 The Software Process

A process is a collection of activities, actions, and tasks that are performed when some work product is to be created. An activity strives to achieve a broad objective (e.g., communication with stakeholders) and is applied regardless of the application domain, size of the project, complexity of the effort, or degree of rigor with which software engineering is to be applied. An action (e.g., architectural design) encompasses a set of tasks that produce a major work product (e.g., an architectural design model). A task focuses on a small, but well-defined objective (e.g., conducting a unit tests) that produces a tangible outcome.

A process framework establishes the foundation for a complete software engineering process by identifying a small number of framework activities that are applicable to all software projects, regardless of their size or complexity. In addition, the process framework encompasses a set of umbrella activities that are applicable across the entire software process. A generic process framework for software engineering encompasses five activities:

Communication: Before any technical work can commence, it is critically important to communicate and collaborate with the customer (and other stakeholders). The intent is to understand stakeholders' objectives for the project and to gather requirements that help define software features and functions.

Planning: Any complicated journey can be simplified if a map exists. A software project is a complicated journey, and the planning activity creates a map that helps guide the team as it makes the journey. The map, called a software project plan, defines the software engineering work by describing the technical tasks to be conducted, the risks that are likely, the resources that will be required, the work products to be produced, and a work schedule.

Modelling: Whether you're a landscaper, a bridge builder, an aeronautical engineer, a carpenter, or an architect, you work with models every day. You create a sketch of the thing so that you'll understand the big picture, what it will look like architecturally, how the constituent parts fit together, and many other characteristics. If required, you refine the sketch into greater and greater detail in an effort to better understand the problem and how you're going to solve it. A software engineer does the same thing by creating models to better understand software requirements and the design that will achieve those requirements.

Construction: This activity combines code generation (either manual or automated) and the testing that is required to uncover errors in the code.

Deployment: The software (as a complete entity or as a partially completed increment) is delivered to the customer who evaluates the delivered product and provides feedback based on the evaluation. Software engineering process framework activities are complemented by a number of umbrella activities. In general, umbrella activities are applied throughout a software project and help a software team manage and control progress, quality, change, and risk. **Typical umbrella activities include:**

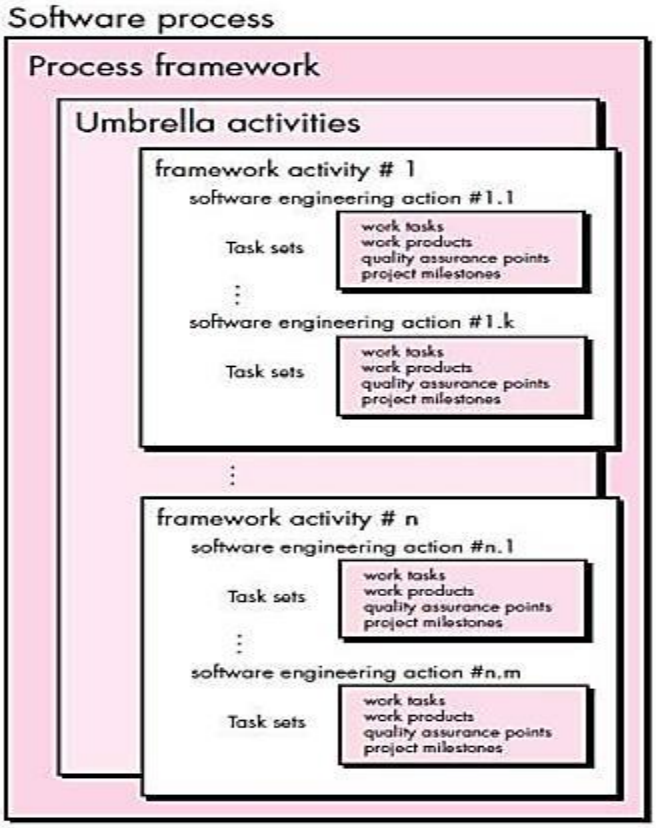
• Software project tracking and control— It allows the software team to assess progress against the project plan and take any necessary action to maintain the schedule.

• Risk management— It assesses risks that may affect the outcome of the project or the quality of the product.

• Software quality assurance— it defines and conducts the activities required to ensure software quality.

• Technical reviews— it assess software engineering work products in an effort to uncover and remove errors before they are propagated to the next activity.

• Measurement— It defines and collects process, project, and product measures that assist the team in delivering software that meets stakeholders' needs;
• Software configuration management— It manages the effects of change throughout the software process.
• Reusability management— It defines criteria for work product reuse (including software components) and establishes mechanisms to achieve reusable components.
• Work product preparation and production—encompasses the activities required to create work products such as models, documents, logs, forms, and lists.



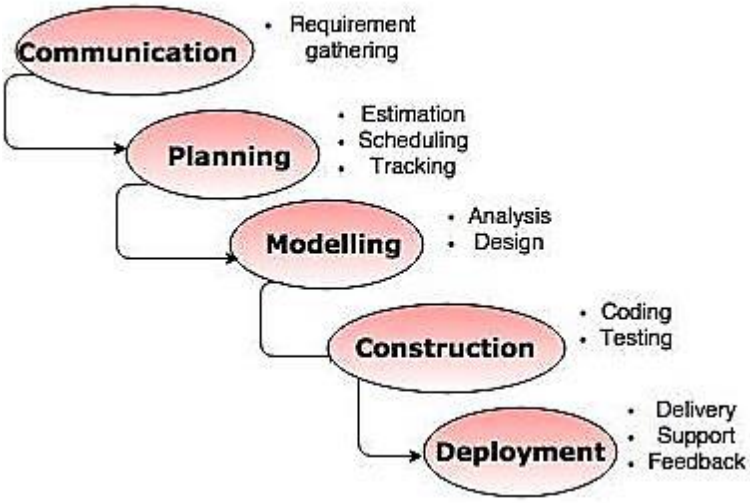
1.3 Prescriptive Process Models

Prescriptive process models were originally proposed to bring order to the chaos of software development. History has indicated that these traditional models have brought a certain amount of useful structure to software engineering work and have provided a reasonably effective road map for software teams. Each process model also prescribes a process flow (also called a work flow) that is, the manner in which the process elements are interrelated to one another.

1.The Waterfall Model(Linear sequential model)

There are times when the requirements for a problem are well understood, when work flows from communication through deployment in a reasonably linear fashion. This situation is sometimes encountered when well-defined adaptations or enhancements to an existing system must be made (e.g., an adaptation to accounting software that has been mandated because of changes to government regulations). It may also occur in a limited number of new development efforts, but only when requirements are well defined and reasonably stable.

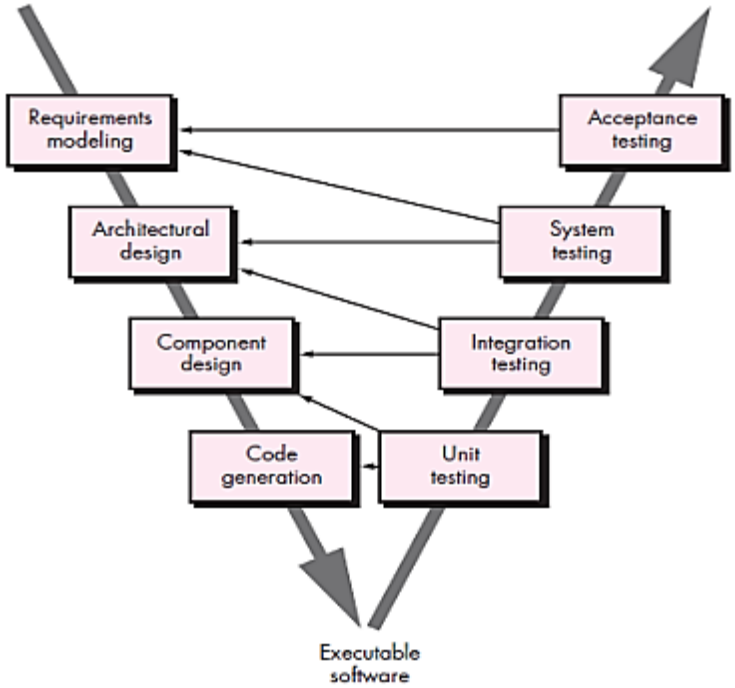
The waterfall model, sometimes called the classic life cycle, suggests a systematic, sequential approach to software development that begins with customer specification of requirements and progresses through planning, modelling, construction, and deployment, culminating in ongoing support of the completed software.



2. V-model

A variation in the representation of the waterfall model is called the V-model. The V-model depicts the relationship of quality assurance actions to the actions associated with communication, modelling, and early construction activities.

As software team moves down the left side of the V, basic problem requirements are refined into progressively more detailed and technical representations of the problem and its solution. Once code has been generated, the team moves up the right side of the V, essentially performing a series of tests (quality assurance actions) that validate each of the models created as the team moved down the left side.In reality, there is no fundamental difference between the classic life cycle and the V-model.



The waterfall model is the oldest paradigm for software engineering. Among the problems that are sometimes encountered when the waterfall model is applied are:

• Real projects rarely follow the sequential flow that the model proposes. Although the linear model can accommodate iteration, it does so indirectly. As a result, changes can cause confusion as the project team proceeds.

• It is often difficult for the customer to state all requirements explicitly. The waterfall model requires this and has difficulty accommodating the natural uncertainty that exists at the beginning of many projects.

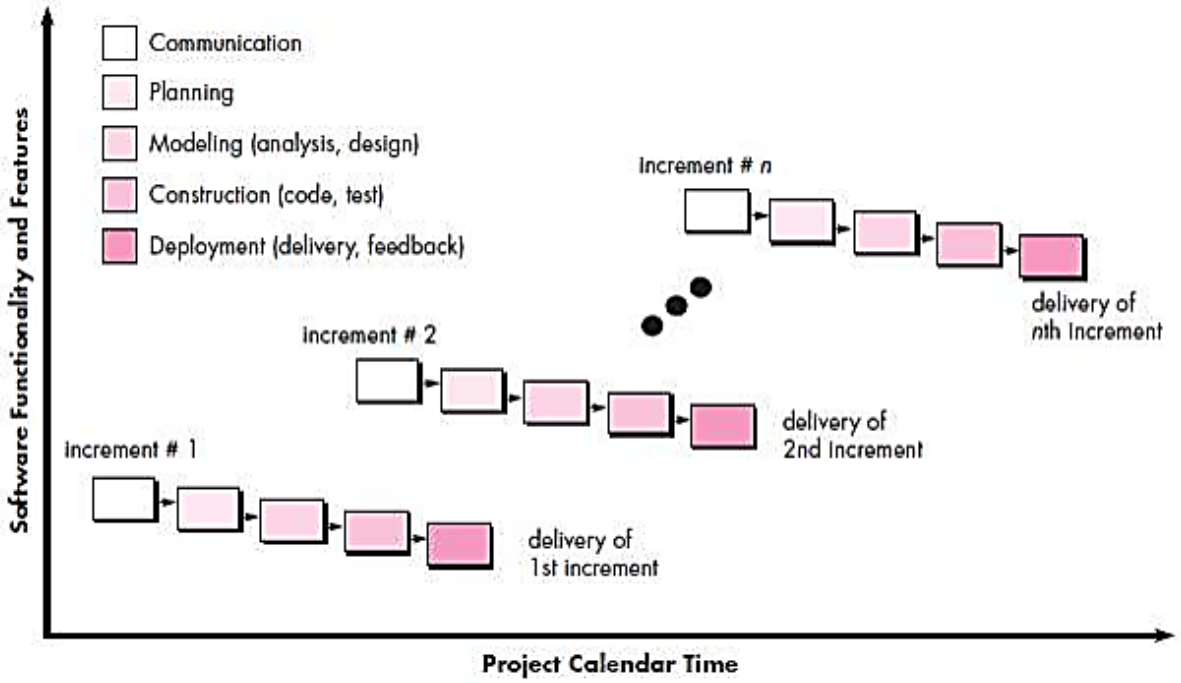
• The customer must have patience. A working version of the program(s) will not be available until late in the project time span. A major blunder, if undetected until the working program is reviewed, can be disastrous.

3. The Incremental Model

The incremental model combines elements of linear and parallel process flows. The incremental model applies linear sequences in a staggered fashion as calendar time progresses. Each linear sequence produces deliverable "increments" of the software in a manner that is similar to the increments produced by an evolutionary process flow.

For example, word-processing software developed using the incremental paradigm might deliver basic file management, editing, and document production functions in the first increment; more sophisticated editing and document production capabilities in the second increment; spelling and grammar checking in the third increment; and advanced page layout capability in the fourth increment. It should be noted that the process flow for any increment can incorporate the prototyping paradigm.

When an incremental model is used, the first increment is often a core product. That is, basic requirements are addressed but many supplementary features (some known, others unknown) remain undelivered. The core product is used by the customer (or undergoes detailed evaluation). As a result of use and/or evaluation, a plan is developed for the next increment.



The plan addresses the modification of the core product to better meet the needs of the customer and the delivery of additional features and functionality. This process is repeated following the delivery of each increment, until the complete product is produced.

The incremental process model focuses on the delivery of an operational product with each increment. Early increments are stripped-down versions of the final product, but they do provide capability that serves the user and also provide a platform for evaluation by the user. Incremental development is particularly useful when staffing is unavailable for a complete implementation by the business deadline that has been established for the project. Early increments can be implemented with fewer people. If the core product is well received, then additional staff (if required) can be added to implement the next increment. In addition, increments can be planned to manage technical risks.

4. Evolutionary Process Models

Software, like all complex systems, evolves over a period of time. Business and product requirements often change as development proceeds, making a straight line path to an end product unrealistic; Two common evolutionary process models are,

• Prototyping

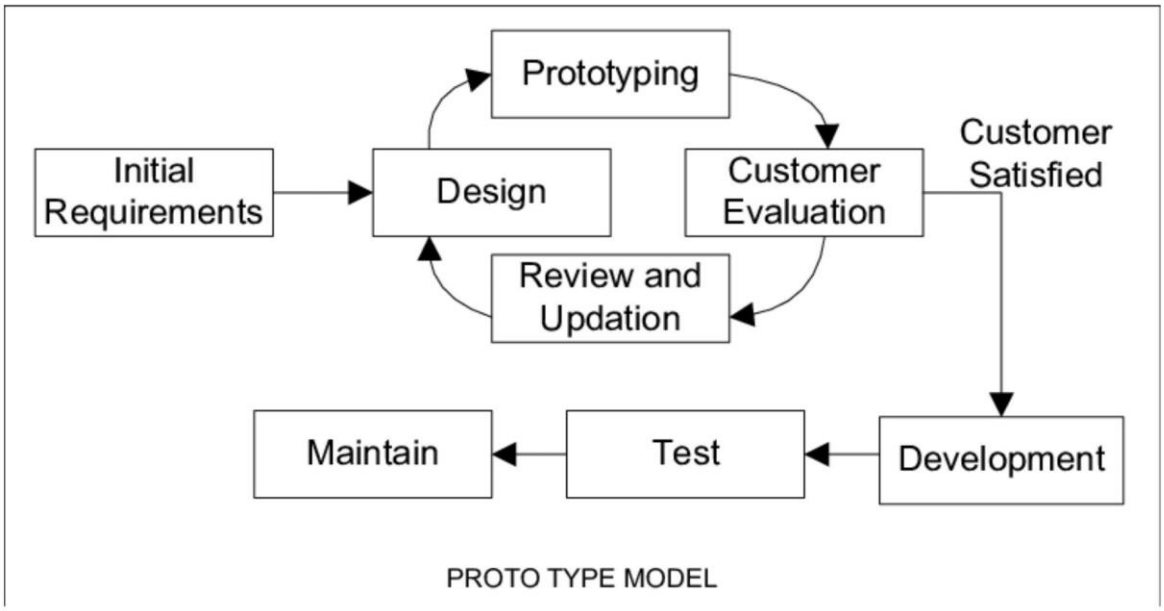
• Spiral Model

Prototyping

The prototyping paradigm begins with communication. You meet with other stakeholders to define the overall objectives for the software, identify whatever requirements are known, and outline areas where further definition is mandatory. Prototyping iteration is planned quickly, and modelling occurs.

A quick design focuses on a representation of those aspects of the software that will be visible to end users (e.g., human interface layout or output display formats). The quick design leads to the construction of a prototype. The prototype is deployed and evaluated by stakeholders, who provide feedback that is used to further refine requirements.

Iteration occurs as the prototype is tuned to satisfy the needs of various stakeholders, while at the same time enabling you to better understand what needs to be done. Ideally, the prototype serves as a mechanism for identifying software requirements.



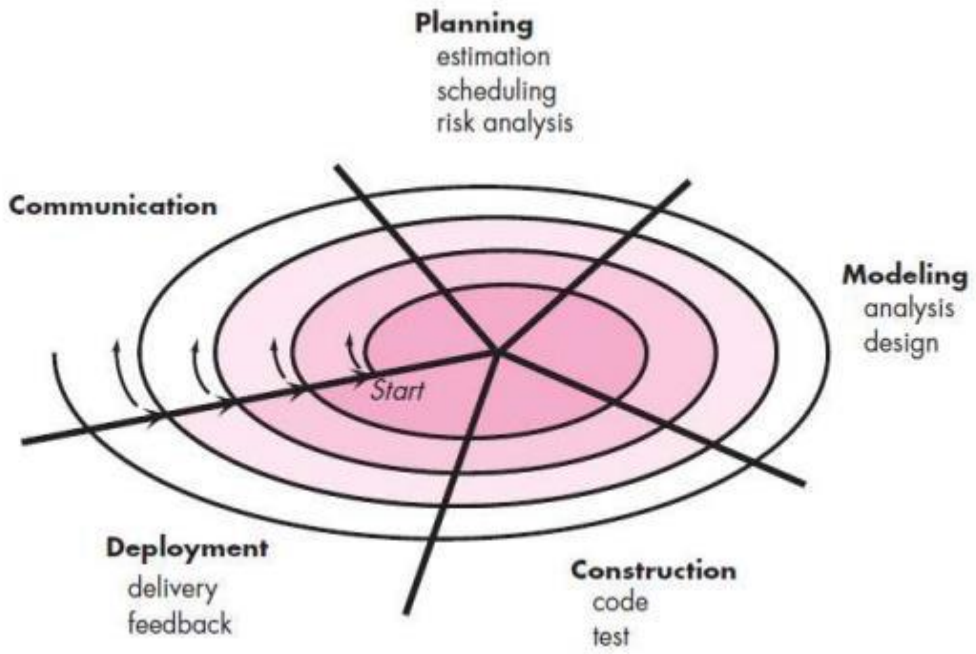
PROTO TYPE MODEL

If a working prototype is to be built, you can make use of existing program fragments or apply tools (e.g., report generators and window managers) that enable working programs to be generated quickly.

5. The Spiral Model

It was originally proposed by Barry Boehm, the spiral model is an evolutionary software process model that couples the iterative nature of prototyping with the controlled and systematic aspects of the waterfall model. It provides the potential for rapid development of increasingly more complete versions of the software.

A spiral model is divided into a set of framework activities defined by the software engineering team. Each of the framework activities represents one segment of the spiral path. As this evolutionary process begins, the software team performs activities that are implied by a circuit around the spiral in a clockwise direction, beginning at the centre.

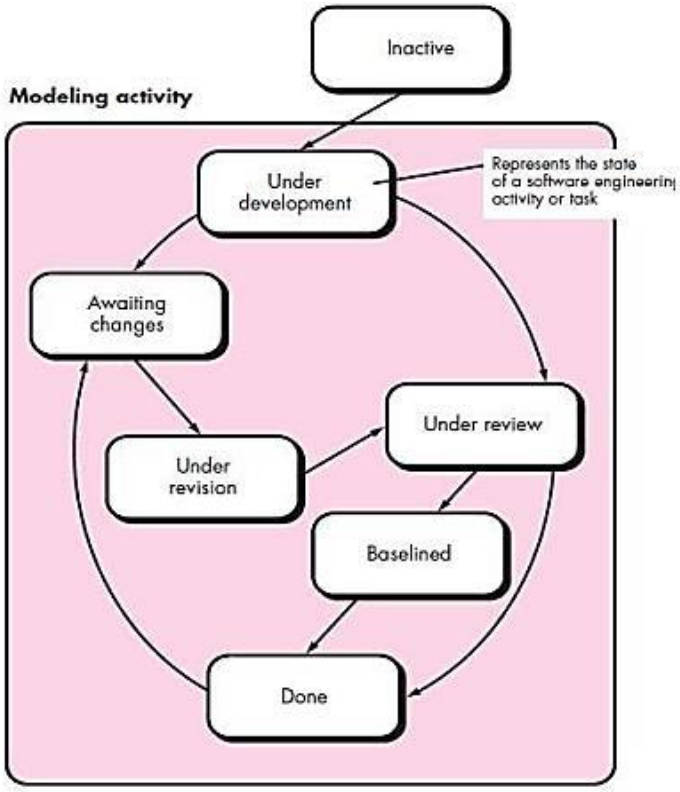


Risk is considered as each revolution is made. Anchor point milestones a combination of work products and conditions that are attained along the path of the spiral are noted for each evolutionary pass. The first circuit around the spiral might result in the development of a product specification; subsequent passes around the spiral might be used to develop a prototype and then progressively more sophisticated versions of the software. Each pass through the planning region results in adjustments to the project plan. Cost and schedule are adjusted based on feedback derived from the customer after delivery.

In addition, the project manager adjusts the planned number of iterations required to complete the software.

6. Concurrent Models

The concurrent development model, sometimes called concurrent engineering, allows a software team to represent iterative and concurrent elements of any of the process models. For example, the modelling activity defined for the spiral model is accomplished by invoking one or more of the following software engineering actions: prototyping, analysis, and design. All software engineering activities exist concurrently but reside in different states. The modelling activity (which existed in the inactive state while initial communication was completed, now makes a transition into the under development state.



If, however, the customer indicates that changes in requirements must be made, the modelling activity moves from the under development state into the awaiting changes state. Concurrent modelling defines a series of events that will trigger transitions from state to state for each of the software engineering activities, actions, or tasks.

7. RAD (Rapid Application Development model)

• Using the RAD model, software product is developed in a short period of time.

• The initial activity starts with the communication between customer and developer.

• Planning depends upon the initial requirements and then the requirements are divided into groups.

• Planning is more important to work together on different modules.

The RAD model consist of following phases 1. Business Modelling

• Business modelling consist of the flow of information between various functions in the project.

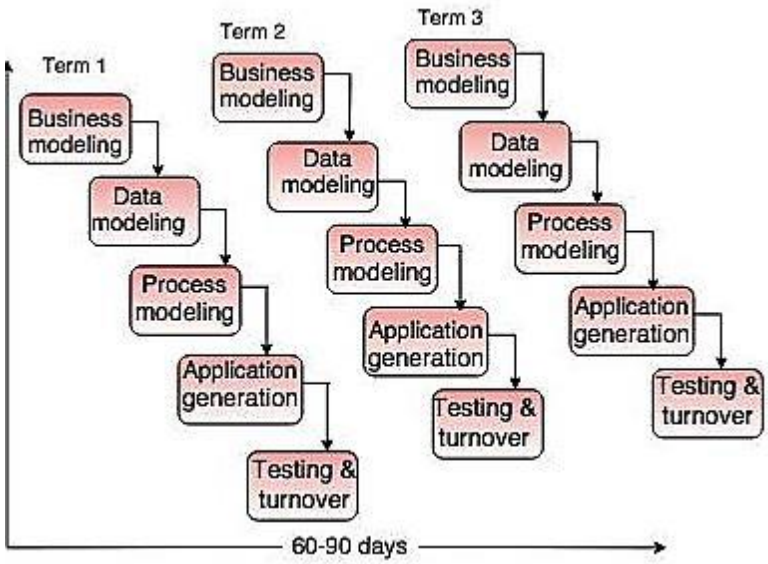
• For example what type of information is produced by every function and which are the functions to handle that information.

• A complete business analysis should be performed to get the essential business information.

2. Data modelling

• The information in the business modelling phase is refined into the set of objects and it is essential for the business.

• The attributes of each object are identified and define the relationship between objects.



3. Process modelling

• The data objects defined in the data modelling phase are changed to fulfil the information flow to implement the business model.

• The process description is created for adding, modifying, deleting or retrieving a data object.

4. Application generation

• In the application generation phase, the actual system is built.

• To construct the software the automated tools are used.

5. Testing and turnover

• The prototypes are independently tested after each iteration so that the overall testing time is reduced.

• The data flow and the interfaces between all the components are fully tested. Hence, most of the programming components are already tested.

1.4 Specialized Process Models

These models tend to be applied when a specialized or narrowly defined software engineering approach is chosen.

1.4.1Component-Based Development

Commercial off-the-shelf (COTS) software components, developed by vendors who offer them as products, provide targeted functionality with well-defined interfaces that enable the component to be integrated into the software that is to be built. The component-based development model incorporates many of the characteristics of the spiral model. It is evolutionary in nature demanding an iterative approach to the creation of software. However, the component-based development model constructs applications from pre-packaged software components. Modelling and construction activities begin with the identification of candidate components. These components can be designed as either conventional software modules or object-oriented classes or packages16 of classes.

The component-based development model incorporates the following steps :

• Available component-based products are researched and evaluated for the

• Application domain in question.

• Component integration issues are considered.

• Software architecture is designed to accommodate the components.

• Components are integrated into the architecture.

• Comprehensive testing is conducted to ensure proper functionality.

The component-based development model leads to software reuse, and reusability.

1.4.2 The Formal Methods Model(FMM)

The formal methods model encompasses a set of activities that leads to formal mathematical specification of computer software. Formal methods enable you to specify, develop, and verify a computer-based system by applying a rigorous, mathematical notation. A variation on this approach, called clean room software engineering is currently applied by some software development organizations.

When formal methods are used during development, they provide a mechanism for eliminating many of the problems that are difficult to overcome using other software engineering paradigms. When formal methods are used during design, they serve as a basis for program verification and therefore enable you to discover and correct errors that might otherwise go undetected. Although not a mainstream approach, the formal methods model offers the promise of defect free software. Yet, concern about its applicability in a business environment has been voiced:

The development of formal models is currently quite time consuming and expensive.

• Because few software developers have the necessary background to apply.

• Formal methods, extensive training is required.

• It is difficult to use the models as a communication mechanism for technically unsophisticated customers.

1.4.3 Aspect-Oriented Software Development

As modern computer-based systems become more sophisticated (and complex), certain concerns—customer required properties or areas of technical interest span the entire architecture. Some concerns are high-level properties of a system (e.g.,s ecurity, fault tolerance). Other concerns affect functions (e.g., the application of business rules), while others are systemic (e.g., task synchronization or memory management).

When concerns cut across multiple system functions, features, and information, they are often referred to as crosscutting concerns. Aspectual requirements define those crosscutting concerns that have an impact across the software architecture. Aspect-oriented software development (AOSD), often referred to as aspect-oriented programming (AOP), is a relatively new software engineering paradigm that provides a process and methodological approach for defining, specifying, designing, and constructing aspects “mechanisms” beyond subroutines and inheritance for localizing the expression of a crosscutting concern .

1.5 Introduction to Agility

Agility has become today's buzzword when describing a modern software process. Everyone is agile. An agile team is a nimble team able to appropriately respond to changes. Changes in the software being built, changes to the team members, changes because of new technology, changes of all kinds that may have an impact on the product they build or the project that creates the product. Support for changes should be built-in everything we do in software, something We embrace because it is the heart and soul of software. An agile team recognizes that software is developed by individuals working in teams and that the skills of these people, their ability to collaborate is at the core for the success of the project. In Jacobson's view, the pervasiveness of change is the primary driver for agility. Software engineers must be quick on their feet if they are to accommodate the rapid changes that Jacobson describes Agility as,

It encourages team structures and attitudes that make communication (among team members, between technologists and business people, between software engineers and their managers) more facile.

It emphasizes rapid delivery of operational software and de-emphasizes the importance of intermediate work products; it adopts the customer as a part of the development team and works to eliminate the us and them‖ attitude that continues to pervade many software projects; it recognizes that planning in an uncertain world has its limits and that a project plan must be flexible.

It can be applied to any software process. However, to accomplish this, it is essential that the process be designed in a way that allows the project team to adapt tasks and to streamline them, conduct planning in a way that understands the fluidity of an agile development approach, eliminate all but the most essential work products and keep them lean, and emphasize an incremental delivery strategy that gets working software to the customer as rapidly as feasible for the product type and operational environment.

1.6 Agile Process

Any agile software process is characterized in a manner that addresses a number of key assumptions about the majority of software projects:

• It is difficult to predict in advance which software requirements will persist and which will change. It is equally difficult to predict how customer priorities will change as the project proceeds.

• For many types of software, design and construction are interleaved. That is, both activities should be performed in tandem so that design models are proven as they are created. It is difficult to predict how much design is necessary before construction is used to prove the design.

• Analysis, design, construction, and testing are not as predictable (from a planning point of view) as we might like.

1.6.1 Agility Principles

The Agile Alliance defines 12 agility principles for those who want to achieve agility:

• Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.

• Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.

• Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.

• Business people and developers must work together daily throughout the project.

• Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.

• The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.

• Working software is the primary measure of progress.

• Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.

• Continuous attention to technical excellence and good design enhances agility.

• Simplicity: the art of maximizing the amount of work not done is essential.

• The best architectures, requirements, and designs emerge from self– organizing teams.

• At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behaviour accordingly.

1.6.2 Human Factors

Agile development focuses on the talents and skills of individuals, molding the process to specific people and teams. The key point in this statement is that the process molds to the needs of the people and team, not the other way around.

If members of the software team are to drive the characteristics of the process that is applied to build software, a number of key traits must exist among the people on an agile team and the team itself.

a) Competence

In an agile development context, "competence" encompasses innate talent, specific software-related skills, and overall knowledge of the process that the team has chosen to apply. Skill and knowledge of process can and should be taught to all people who serve as agile team members.

b) Common focus

Although members of the agile team may perform different tasks and bring different skills to the project, all should be focused on one goal to deliver a working software increment to the customer within the time promised.

c) Collaboration

Software engineering is about,

• Assessing, analysing, and using information that is communicated to the software team; creating information that will help all stakeholders understand the work of the team;

• Building information that provides business value for the customer.

To accomplish these tasks, team members must collaborate with one another and all other stakeholders.

d) Decision-making ability

Any good software team must be allowed the freedom to control its own destiny. This implies that the team is given autonomy—decision-making authority for both technical and project issues.

e) Fuzzy problem-solving ability

Software managers must recognize that the agile team will continually have to deal with ambiguity and will continually be buffeted by change. In some cases, the team must accept the fact that the problem they are solving today may not be the problem that needs to be solved tomorrow.

f) Mutual trust and respect

A jelled team exhibits the trust and respect that are necessary to make them —so strongly knit that the whole is greater than the sum of the parts.

g) Self-organization

In the context of agile development, self-organization implies three things:

• The agile team organizes itself for the work to be done,

• The team organizes the process to best accommodate its local environment,

• The team organizes the work schedule to best achieve delivery of the software increment.

1.7 Extreme Programming (Xp)

The most widely used approach to agile software development are,

1.7.1 XP Values

Beck defines a set of five values that establish a foundation for all work performed as part of XP communication, simplicity, feedback, courage, and respect. Each of these values is used as a driver for specific XP activities, actions, and tasks.

In order to achieve effective communication between software engineers and other stakeholders XP emphasizes close, yet informal (verbal) collaboration between customers and developers, the establishment of effective metaphors3 for communicating important concepts, continuous feedback, and the avoidance of voluminous documentation as a communication medium.

To achieve simplicity, XP restricts developers to design only for immediate needs, rather than consider future needs. The intent is to create a simple design that can be easily implemented in code. If the design must be improved, it can be refactored at a later time.

• Feedback is derived from three sources:

• the implemented software itself,

• the customer, and

• Other software team members.

By designing and implementing an effective testing strategy the software (via test results) provides the agile team with feedback. XP makes use of the unit test as its primary testing tactic. As each class is developed, the team develops a unit test to exercise each operation according to its specified functionality.

As an increment is delivered to a customer, the user stories or use cases that are implemented by the increment are used as a basis for acceptance tests. The degree to which the software implements the output, function, and behavior of the use case is a form of feedback. Finally, as new requirements are derived as part of iterative planning, the team provides the customer with rapid feedback regarding cost and schedule impact.

1.7.2 The XP Process

Extreme Programming uses an object-oriented approach as it's a preferred development paradigm and encompasses a set of rules and practices that occur within the context of four framework activities: planning, design, coding, and testing.

Key XP activities are summarized as:

a) Planning

The planning activity (also called the planning game) begins with listening a requirements gathering activity that enables the technical members of the XP team to understand the business context for the software and to get a broad feel for required output and major features and functionality.

Listening leads to the creation of a set of ―stories‖ (also called user stories) that describe required output, features, and functionality for software to be built. Each story is written by the customer and is placed on an index card. The customer assigns a value (i.e., a priority) to the story based on the overall business value of the feature or function.5 Members of the XP team then assess each story and assign a cost measured in development weeks to it. If the story is estimated to require more than three development weeks, the customer is asked to split the story into smaller stories and the assignment of value and cost occurs again.

It is important to note that new stories can be written at any time. Customers and developers work together to decide how to group stories into the next release (the next software increment) to be developed by the XP team. Once a basic commitment (agreement on stories to be included, delivery date, and other project matters) is made for a release, the XP team orders the stories that will be developed in one of three ways:

• All stories will be implemented immediately (within a few weeks),

• The stories with highest value will be moved up in the schedule and implemented first, or

• The riskiest stories will be moved up in the schedule and implemented first.

After the first project release (also called a software increment) has been delivered, the XP team computes project velocity. Stated simply, project velocity is the number of customer stories implemented during the first release. Project velocity can then be used to

• Help estimate delivery dates and schedule for subsequent releases and

• Determine whether an over commitment has been made for all stories across the entire development project.

As development work proceeds, the customer can add stories, change the value of an existing story, split stories, or eliminate them. The XP team then reconsiders all remaining releases and modifies its plans accordingly.

b) Design

XP design rigorously follows the KIS (keep it simple) principle. A simple design is always preferred over a more complex representation. In addition, the design provides implementation guidance for a story as it is written nothing less, nothing more.

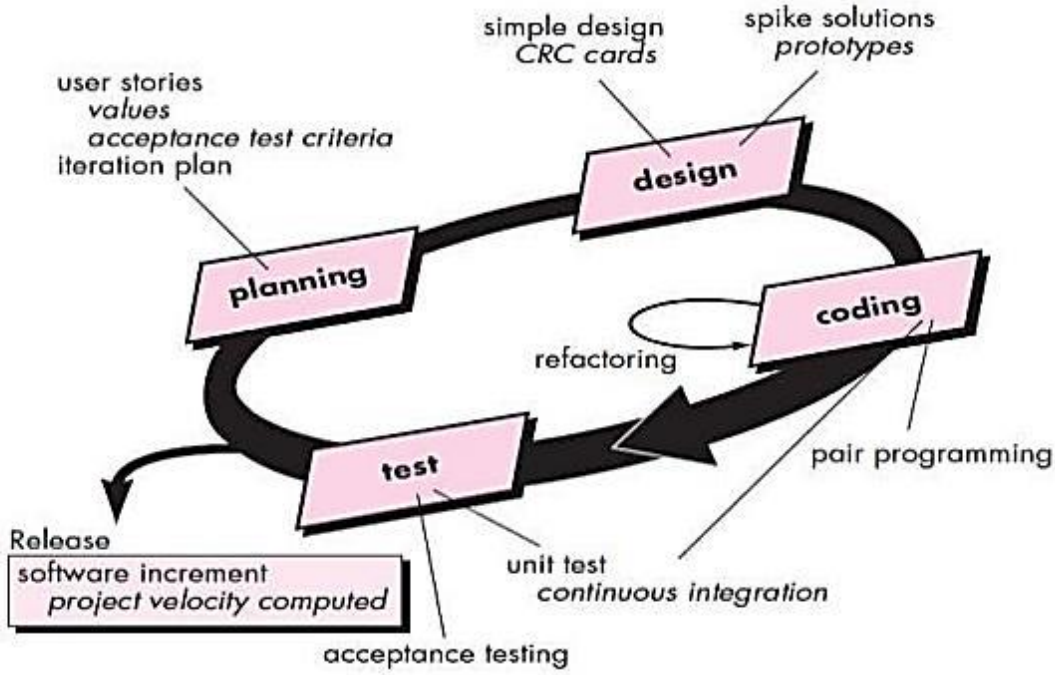
c) Coding

After stories are developed and preliminary design work is done, the team does not move to code, but rather develops a series of unit tests that will exercise each of the stories that is to be included in the current release. Once the unit test has been created, the developer is better able to focus on what must

be implemented to pass the test. Nothing extraneous is added (KIS). Once the code is complete, it can be unit-tested immediately, thereby providing instantaneous feedback to the developers.

d) Testing

The unit tests that are created should be implemented using a framework that enables them to be automated (hence, they can be executed easily and repeatedly). This encourages a regression testing strategy whenever code is modified (which is often, given the XP refactoring). As the individual unit tests are organized into a ―universal testing suite‖ integration and validation testing of the system can occur on a daily basis. This provides the XP team with a continual indication of progress and also can raise warning flags early if things go away.



1.7.3 Industrial XP

IXP is an organic evolution of XP. It is imbued with XP's minimalist, customer-centric, test-driven spirit. IXP differs most from the original XP in its greater inclusion of management, its expanded role for customers, and its upgraded technical practices.‖ IXP incorporates six new practices that are designed to help ensure that an XP project works successfully for significant projects within a large organization. Readiness assessment: Prior to the initiation of an IXP project, the organization should conduct a readiness assessment. The assessment ascertains whether

• an appropriate development environment exists to support IXP,

• the team will be populated by the proper set of stakeholders, the organization has a distinct quality program and supports continuous improvement,

• the organizational culture will support the new values of an agile team, and

• The broader project community will be populated appropriately.

Project community

Classic XP suggests that the right people be used to populate the agile team to ensure success. The implication is that people on the team must be well-trained, adaptable and skilled, and have the proper temperament to contribute to a self-organizing team. When XP is to be applied for a significant project in a large organization, the concept of the "team" should morph into that of a community.

A community may have a technologist and customers who are central to the success of a project as well as many other stakeholders (e.g., legal staff, quality auditors, manufacturing or sales types) who ―are often at the periphery of an IXP project yet they may play important roles on the project‖. In IXP, the community members and their roles should be explicitly defined and mechanisms for communication and coordination between community members should be established.

Project chartering

The IXP team assesses the project itself to determine,

• Whether an appropriate business justification for the project exists and

• Whether the project will further the overall goals and objectives of the organization.

Chartering also examines the context of the project to determine how it complements, extends, or replaces existing systems or processes.

Test-driven management

An IXP project requires measurable criteria for assessing the state of the project and the progress that has been made to date. Test-driven management establishes a series of measurable "destinations" and then defines mechanisms for determining whether or not these destinations have been reached.

Retrospectives

An IXP team conducts a specialized technical review after a software increment is delivered. It is called a retrospective, the review examines "issues", events, and "lessons-learned" across a software increment and/or the entire software release. The intent is to improve the IXP process.

Continuous learning

Because learning is a vital part of continuous process improvement, members of the XP team are encouraged to learn new methods and techniques that can lead to a higher quality product. In addition to the six new practices discussed, IXP modifies a number of existing XP practices. Story-driven development (SDD) insists that stories for acceptance tests be written before a single line of code is generated. Domain-driven design (DDD) is an improvement on the "system metaphor" concept used in XP. DDD suggests the evolutionary creation of a domain model that "accurately represents how domain experts think about their subject"

1.7.4 The XP Debate

All new process models and methods spur worthwhile discussion and in some instances heated debate .Among the issues that continue to trouble some critics of XP are:

• Requirements volatility: Because the customer is an active member of the XP team, changes to requirements are requested informally. As a consequence, the scope of the project can change and earlier work may have to be modified to accommodate current needs.

• Conflicting customer needs: Many projects have multiple customers, each with his own set of needs. In XP, the team itself is tasked with assimilating the needs of different customers, a job that may be beyond their scope of authority.

• **Requirements are expressed informally:** User stories and acceptance tests are the only explicit manifestation of requirements in XP. Critics argue that a more formal model or specification is often needed to ensure that omissions, inconsistencies, and errors are uncovered before the system is built. Proponents counter that the changing nature of requirements makes such models and specification obsolete almost as soon as they are developed.

• **Lack of formal design:** XP deemphasizes the need for architectural design and in many instances, suggests that design of all kinds should be relatively informal.

Part A (Two Marks)

1. **What is software engineering?**

Software engineering is a discipline in which theories, methods and tools are applied to develop professional software.

2. **What is Software?**

Software is nothing but a collection of computer programs that are related documents that are indented to provide desired features, functionalities and better performance.

3. **What are the characteristics of the software?**

• Software is engineered, not manufactured.

• Software does not wear out.

• Most software is custom built rather than being assembled from components.

4. **What are the various categories of software?**

• System software

• Application software

• Engineering/Scientific software

• Embedded software

5. **What are the challenges in software?**

• Copying with legacy systems.

• Heterogeneity challenge

• Delivery times challenge.

6. **Define software process**.

Software process is defined as the structured set of activities that are required to develop the software system.

7. **What are the fundamental activities of a software process?**

• Specification

• Design and implementation

• Validation

• Evolution

8. **What are the umbrella activities of a software process?**

• Software project tracking and control.

• Risk management.

• Software Quality Assurance.

• Formal Technical Reviews.

• Software Configuration Management.

• Work product preparation and production.

• Reusability management.

• Measurement.

9. **What are the merits of incremental model?**

• The incremental model can be adopted when there is less number of people involved in the project.

• Technical risks can be managed with each increment.

• For a very small time span, at least core product can be delivered to the customer.

10. **List the task regions in the Spiral model.**

• Customer communication -it is suggested to establish customer communication.

• Planning –All planning activities are carried out

• Risk analysis –The tasks required to calculate technical and management risks.

• Engineering –tasks required to build one or more representations of applications

• Construct and release –tasks required to construct, test, install the applications

• Customer evaluation -tasks are performed and implemented at installation stage based on the customer evaluation.

11. **What are the drawbacks of spiral model?**

• • It is based on customer communication. If the communication is not proper then the software product that gets developed will not be the up to the mark.

• • It demands considerable risk assessment. If the risk assessment is done properly then only the successful product can be obtained.

12. **What is System Engineering?**

System Engineering means designing, implementing, deploying and operating systems which include hardware, software and people.

13. **What is an effector process?**

The effector process is a process that verifies itself. The effector process exists in certain criteria.

14. **What is the use of CMM?**

Capability Maturity Model is used in assessing how well an organization processes allow to complete and manage new software projects.

15. **Name the Evolutionary process Models**.

• Prototyping model

• Spiral model

• Concurrent Development

16. **What is meant by Software engineering paradigm?**

The development strategy that encompasses the process, methods and tools and generic phases is often referred to as a process model or software engineering paradigm.

17. **What are the various elements for computer based system?**

i.Software

ii. Hardware

iii.People

iv.Database

v.Documentation

Part B (Possible Questions)

1. Explain iterative waterfall and spiral model for software life cycle and various activities in each phase.

2. Explain about the incremental model.

3. Explain in detail about the life cycle process.

4. Explain in detail about the software process.

5. Explain iterative waterfall and spiral model for software life cycle and discuss various activities in each phase.

6. Explain in detail Boehm's spiral model for software life cycle and discuss various activities in each phase.

7. Distinguish between verification and validation process.

8. Discuss about the layers of software engineering.

Part C (Possible Questions)

1. a) Which is more important-the product or process? Justify your answer.

b) Identify the umbrella activities in software engineering process.

c) With suitable illustration explain SPIRAL model evolutionary software development.

2. Explain in detail about incremental and rapid application development Model (RAD) and mention its advantages and disadvantages.

3. Difference between perspective and evolutionary model.